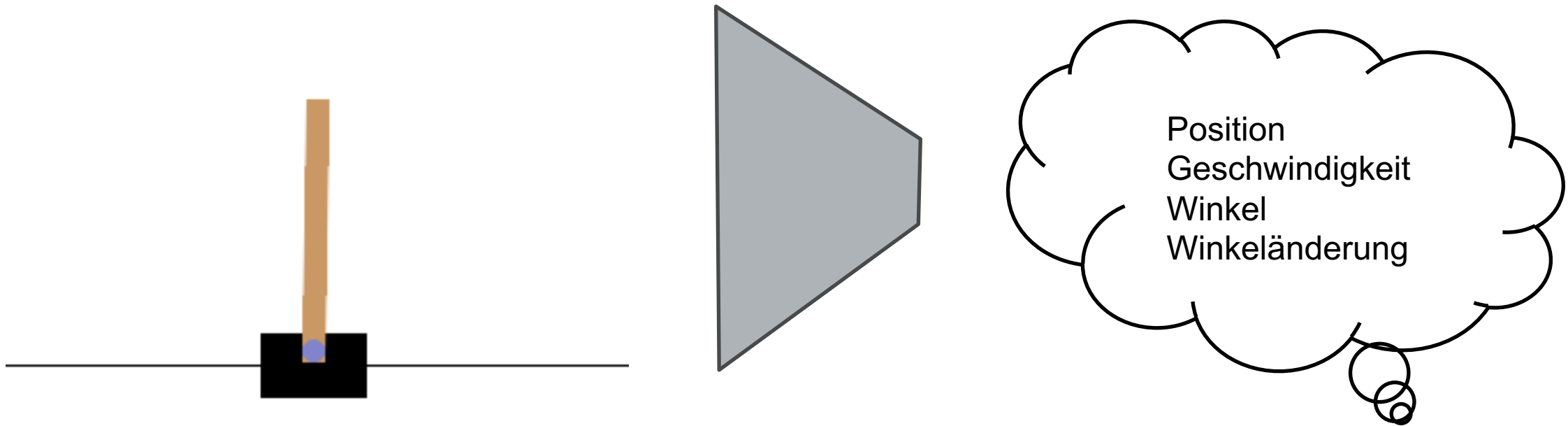




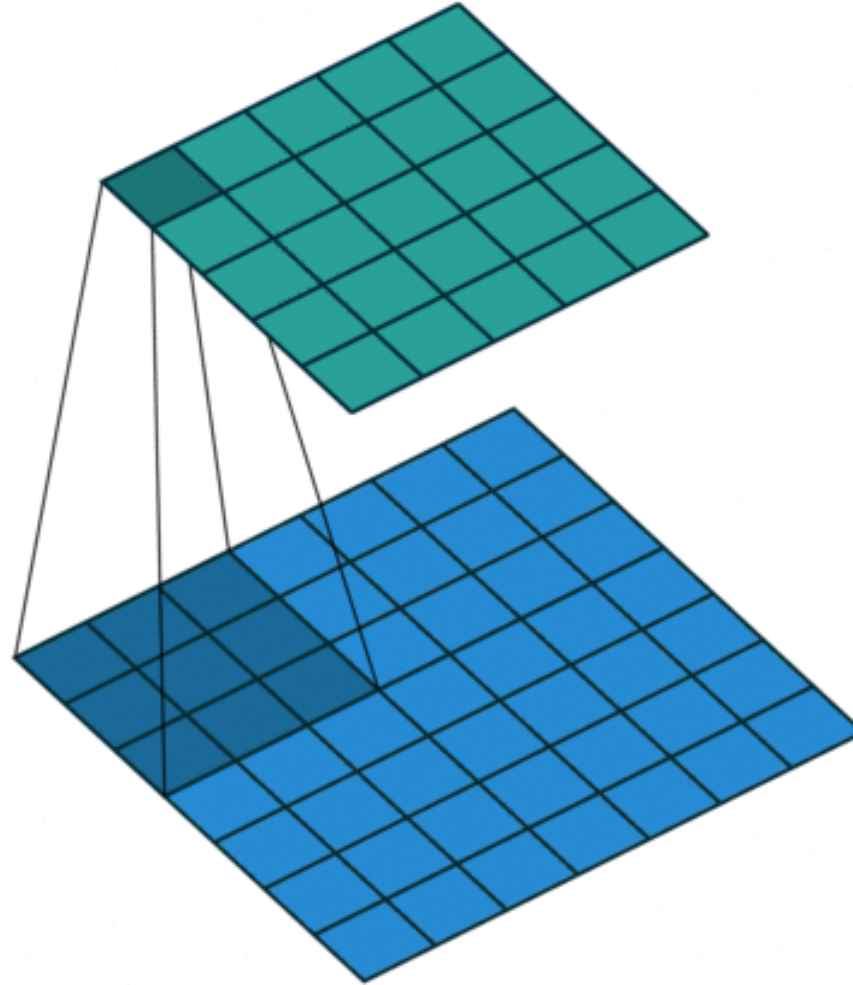
Sprint 4

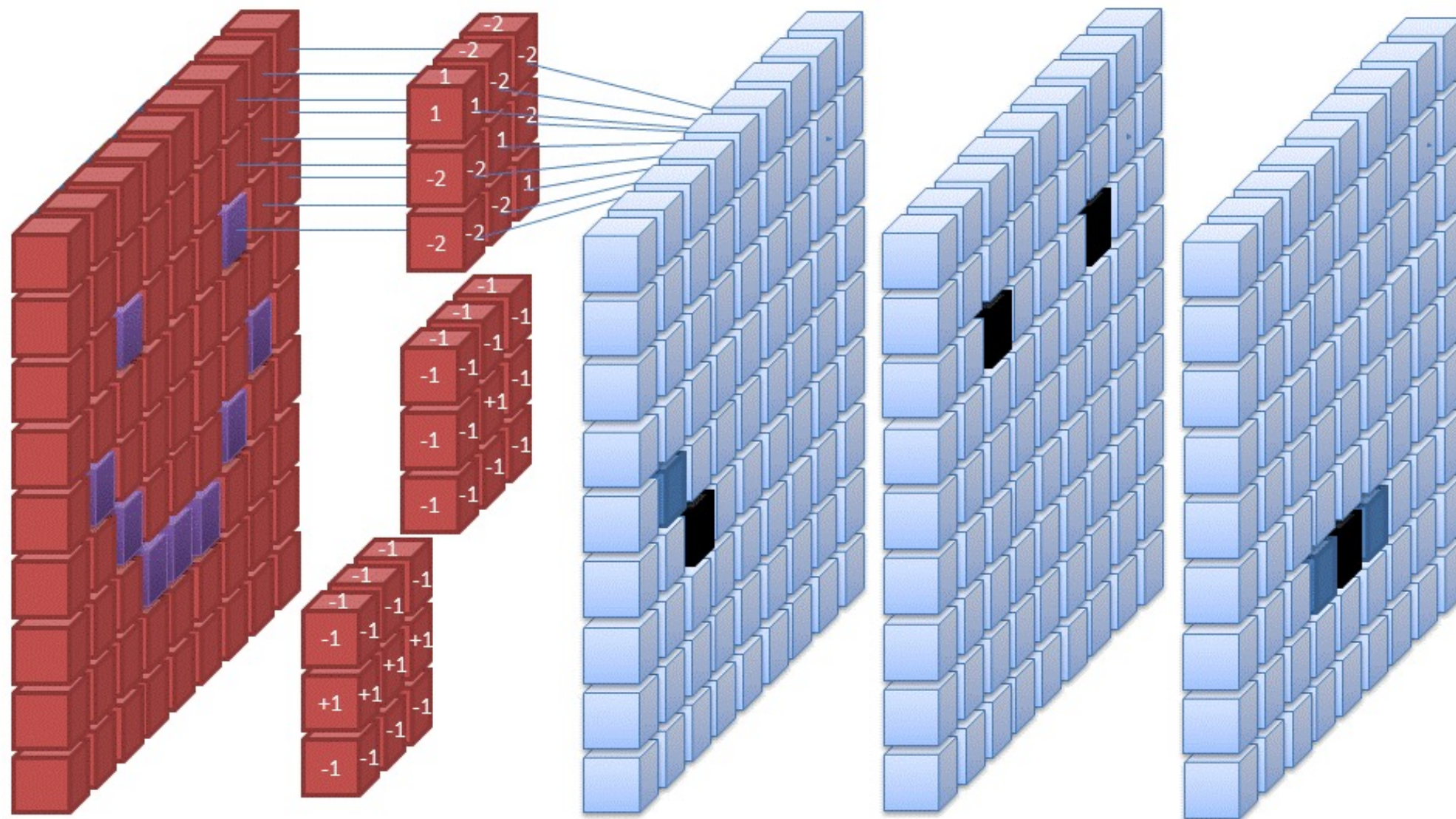
17.06.26





Convolutional NNs







CNN

```
nn.Conv2D(  
    in_channels,  
    out_channels,  
    kernel_size,  
    stride  
)
```



Dreamer vs. MuZero

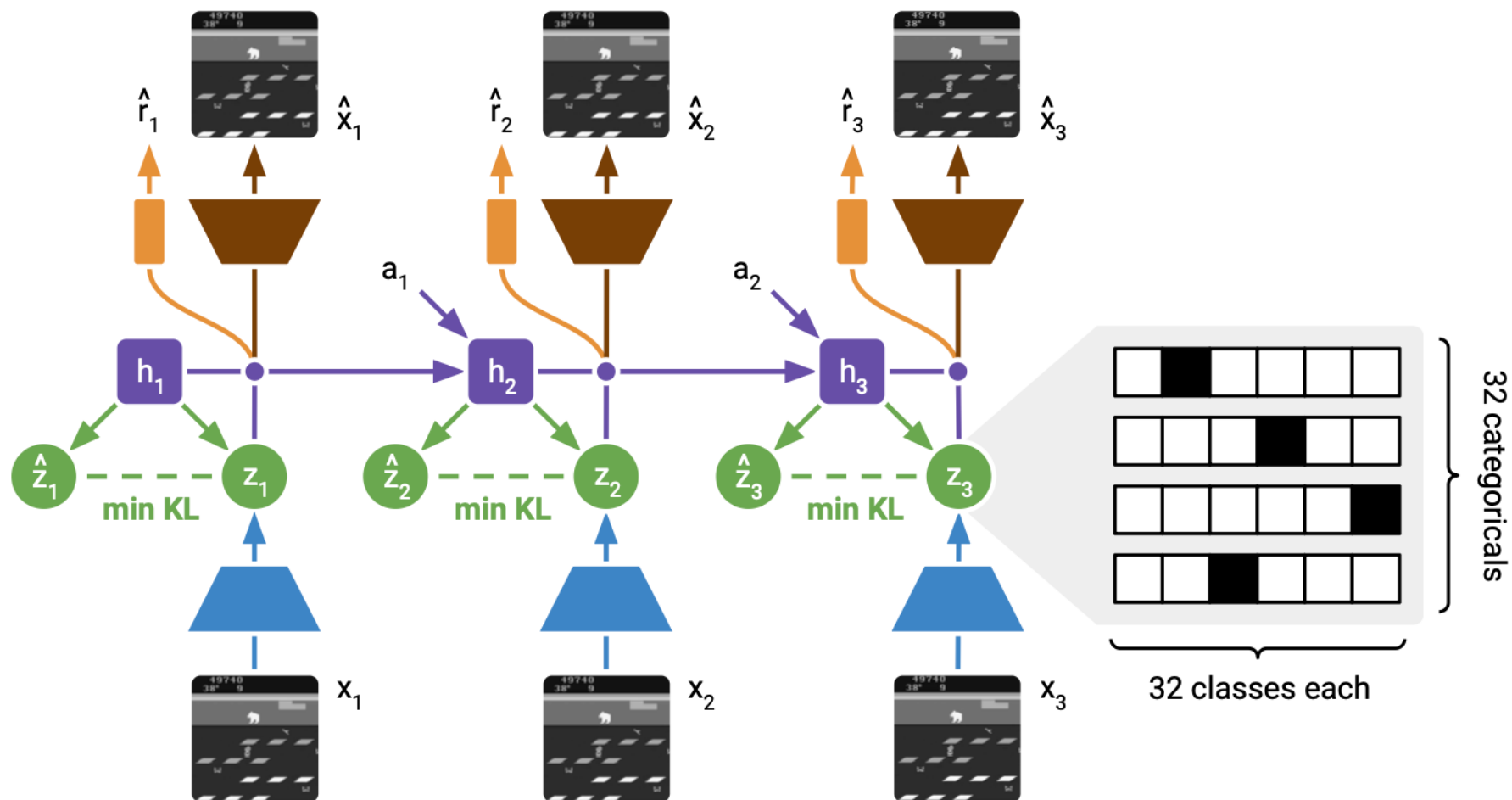
- RNN h_t
- Encoder $x_t \mapsto z_t$
- $s_t \mapsto a_t, v_t, r_t$
- $s, a_t \mapsto s_{t+1}$
- Decoder $s_t \mapsto x_t$

* $s_t = (h_t, z_t)$

Encoder $x_t \mapsto e_t$

Posterior model $e_t, h_t \mapsto z_t$

- Kein RNN
- Representation function $x_{\{t-4, \dots, t\}} \mapsto s_t$
- $s_t \mapsto p_t, v_t$
- $s_t, a_t \mapsto s_{t+1}, r_{t+1}$
- Kein decoder





Alles passiert im Latent Space!

- Dreamer: Simulation (Imagination) im Latent Space
- MuZero: Planung im Latent Space



Wie funktioniert das Training?

- Dreamer (world model learning)
 - Reconstruction loss
 - Decodiertes Bild vs. echtes Bild
 - Reward/continue loss
 - Vorhergesagter reward/continue vs. echter reward/continue
 - KL loss
 - Prior $p(z | h)$ vs. posterior $q(z | h, o)$



Wie funktioniert das Training?

- MuZero

- Policy loss

- Vorhergesagte search policy vs. MCTS visit counts

- Value loss

- Vorhergesagtes value vs. bootstrapped return

- Reward loss

- Vorhergesagter reward vs. echter reward

} Nur aus MCTS mit
dynamics model

=> Representation function wird nur impliziert trainiert



Ziele für Sprint 4

1. Bild generieren mit variabler Auflösung (e.g., 32x32)
2. Neuronale Netzwerke implementieren
 1. Representation function (MuZero)
 2. Encoder/Decoder (Dreamer)
3. Training
4. Latent space analysieren
 1. Können wir Position, Geschwindigkeit, Winkel, Winkelveränderung auslesen?
5. Ziel: Cartpole mit Bild-Input



Architektur (Encoder)

1. Convolutional layer (1 in, 8 out, size 3, stride 2)
2. ReLU
3. Convolutional layer (8 in, 16 out, size 3, stride 2)
4. ReLU
5. Flatten
6. Linear layer ($16 \times 7 \times 7$ in, 64 out)
7. ReLU
8. Linear layer (64 in, 32 out)



Architektur (Representation Funktion)

1. Convolutional layer (4 in, 8 out, size 3, stride 2)
2. ReLU
3. Convolutional layer (8 in, 16 out, size 3, stride 2)
4. ReLU
5. Flatten
6. Linear layer ($16 \times 7 \times 7$ in, 64 out)
7. ReLU
8. Linear layer (64 in, 32 out)



Architektur (Decoder)

1. Linear layer (32 in, 64 out)
2. ReLU
3. Linear layer (64 in, $16 \times 7 \times 7$ out)
4. ReLU
5. Unflatten ($16 \times 7 \times 7$)
6. Transposed Convolutional layer (16 in, 8 out, size 3, stride 2)
7. ReLU
8. Transposed Convolutional layer (8 in, 1 out, size 3, stride 2)